Prácticas de Ampliación de Sistemas Operativos • `minigrep`

Descripción del programa `minigrep`

Escribe un programa llamado `minigrep` que lea un flujo de líneas (secuencias de bytes terminadas en `\n`) de la entrada estándar y muestre en la salida estándar aquellas líneas que contengan una subcadena reconocida por la expresión regular dada. Por ejemplo:

```
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "^[[:digit:]]67[[:digit:]]$"
5678
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "^[[:digit:]]67[[:digit:]]$" -v
1234
90AB
$ ./minigrep -r "^[[:digit:]]67[[:digit:]]$"
1234
5678
90AB
5678
```

Ten en cuenta que en el último ejemplo, se introducen por teclado tres líneas con 1234, 5678 y 90AB respectivamente, pulsando tras el último retorno de carro `Ctrl+D` para indicar final de fichero. La última línea es la salida del programa, como se aprecia en el primer ejemplo.

El programa tiene que aceptar las siguientes opciones (`-n` y `-o` son ampliaciones optativas, pero la ayuda es la misma para todos):

```
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-n] [-o] [-h]
    -r REGEX Expresión regular.
    -s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por defecto,
1024).
    -v Muestra las líneas que NO sean reconocidas por la expresión regular.
    -n Antepone a cada línea mostrada su número de línea en la entrada.
    -o Muestra solo la parte de cada línea reconocida por la expresión regular.
```

La ayuda debe mostrarse **tal cual aparece arriba**: los tests la comprueban carácter a carácter y cada línea de opciones empieza con un tabulador.

Consideraciones, indicaciones y requisitos

- Los argumentos de la línea de órdenes debe procesarse con `getopt()` y no podrá suponerse un orden concreto de dichos argumentos. Además podrá utilizarse más de una vez un argumento, en cuyo caso se tomará la última aparición del mismo como la correcta.

- `REGEX` es una expresión regular. En la Referencia 1 encontrarás cómo compilar expresiones regulares y cómo reconocerlas en una línea de texto. En la Referencia 2 tienes la especificación del formato de las expresiones regulares aceptadas. Se deberán utilizar los flags `REG_NEWLINE` y `REG_EXTENDED` en la llamada a `regcomp()`.

- `BUFSIZE` es el tamaño tanto del buffer de lectura como del buffer de escritura. Ambos buffers tendrán que reservarse en el *heap*. Tanto las lecturas como las escrituras *siempre* intentarán leer o escribir `BUFSIZE` bytes, es decir, leer y escribir la máxima cantidad de bytes que caben en el buffer. Las escrituras parciales deberan tratarse adecuadamente. El tamaño mínimo y máximo de los buffers será de 1 byte y 1 MB, respectivamente.

- Cuando se proporcione la opción `-v` se mostrarán las líneas que no sean reconocidas por la expresión regular en lugar de mostrar aquellas que sí lo sean.

- Para simplificar el problema, supondremos que **las líneas no tendrán más de 4096 bytes**, incluyendo el `\0` de fin de cadena. Si una línea fuera mayor, devolveríamos un error indicándolo e interrumpiríamos la ejecución del programa. Hay que tener en cuenta que para que funcione correctamente el reconocimiento del comienzo y final de línea en las expresiones regulares, no hay que incluir el `\n` final en la cadena. Por tanto, una línea será una secuencia de caracteres, hasta recibir un `\n` o detectar el final de fichero, seguida de un `\0`. El carácter `\n` no se incluirá en la cadena. Sin embargo, al imprimir la cadena en la salida estándar, siempre deberá escribirse un `\n` a continuación. Este es el comportamiento estándar de la orden `grep`. Por ejemplo:

```
$ echo -e -n "1234\n5678" | ./minigrep -r "^[[:digit:]]23[[:digit:]]$"
1234
$ echo -e -n "1234\n5678" | ./minigrep -r "^[[:digit:]]67[[:digit:]]$"
5678
$
```

- La manipulación de archivos debe realizarse con **llamadas al sistema** como, por ejemplo, `open()`, `read()`, `write()` o `close()`. En ningún caso deben usarse las funciones de la biblioteca estándar de C como, por ejemplo, `fopen()`, `fread()`, `fwrite()` o `fclose()`, y similares. Sin embargo, se permite usar `printf()` y `fprintf()` para mostrar mensajes de ayuda o de error.

- Se prohíbe el uso de funciones de manipulación de cadenas como, por ejemplo, `strcpy()` o `strcat()`, y similares. La única excepción es `snprintf()`, que se permite para convertir en texto el número de línea de la opción `-n`.

- El programa `minigrep` lee de la entrada estándar, es decir, debe funcionar correctamente con un **archivo de entrada de tamaño *infinito***. Por tanto, no se puede leer toda la entrada estándar,

procesarla y escribir el resultado en la salida estándar. `minigrep` tiene que intercalar lecturas, procesamiento y escrituras hasta que agote la entrada.

Ejemplos de ejecución de `minigrep`

```
$ ./minigrep -h
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-n] [-o] [-h]
    -r REGEX Expresión regular.
    -s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por defecto,
1024).
    -v Muestra las líneas que NO sean reconocidas por la expresión regular.
    -n Antepone a cada línea mostrada su número de línea en la entrada.
    -o Muestra solo la parte de cada línea reconocida por la expresión regular.

$ echo $?
0
$ ./minigrep -j
./minigrep: invalid option -- 'j'
Uso: ./minigrep -r REGEX [-s BUFSIZE] [-v] [-n] [-o] [-h]
    -r REGEX Expresión regular.
    -s BUFSIZE Tamaño de los buffers de lectura y escritura en bytes (por defecto,
1024).
    -v Muestra las líneas que NO sean reconocidas por la expresión regular.
    -n Antepone a cada línea mostrada su número de línea en la entrada.
    -o Muestra solo la parte de cada línea reconocida por la expresión regular.
$ echo $?
1
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r 'ABCD' -r
"^[[:digit:]]67[[:digit:]]$"
5678
$ echo $?
0
```

Ampliaciones optativas

Las opciones `-n` y `-o` son **optativas**: no hace falta implementarlas para aprobar. Un `minigrep` completo, con las dos, puede sacar un 10; cada una que no se haga **resta 1 punto** a la nota de `minigrep`, pero no suspende. La nota de la práctica es la media de `minigrep` y `watchdir` (50 % cada uno). Por ejemplo, con un 8 en `minigrep` (sin `-n` ni `-o`) y un 10 en `watchdir`, la práctica tiene un 9.

- **La ayuda es la misma para todos**, se implementen o no las ampliaciones: el mensaje de `-h` y el de opción no válida deben incluir `[-n] [-o]` y sus dos líneas de descripción, tal y como aparecen más arriba. Así lo comprueban los tests 1 y 2 de `minigrep.json`.

- **`-n`** antepone a cada línea mostrada su número de línea en la entrada, seguido de `:`, como hace `grep -n`. Por ejemplo, `12:texto`. La numeración empieza en 1 y cuenta **todas** las líneas leídas, también las que no se muestran. Funciona también junto con `-v`.

- **`-o`** muestra solo el trozo de la línea reconocido por la **primera** coincidencia de la expresión regular (el que indica `pmatch[0]` tras `regexec()`, entre `rm_so` y `rm_eo`), seguido de `\n`. A diferencia de `grep -o`,

no se buscan más coincidencias en el resto de la línea. Si la coincidencia es vacía (`rm_so == rm_eo`), por ejemplo con `-r 'x*'`, no se muestra nada para esa línea. Junto con `-n`, el número va delante del trozo: `12:trozo`.

- `-o` junto con `-v` no tiene sentido (no hay trozo reconocido que mostrar) y es un **error**: el programa escribe `ERROR: Las opciones -o y -v son incompatibles` en la salida de error y termina con código de retorno 1. Este error se comprueba después de los demás errores de los argumentos (`REGEX` vacía o mal construida y `BUFSIZE` fuera de rango).

- **Restricción de `-n`**: el número y el `:` forman parte de la salida, así que deben pasar por el **mismo buffer de escritura** que el resto y escribirse por bloques de `BUFSIZE` bytes, tratando las escrituras parciales. Por tanto, el número (un entero en memoria) se convierte en los caracteres de sus cifras decimales, por ejemplo el entero 12 en los caracteres `'1'` y `'2'`, y esos caracteres se meten en el buffer uno a uno. La conversión puede hacerse a mano, cifra a cifra, o con `snprintf()`, la única función de cadenas permitida, sobre un buffer auxiliar. No se puede usar `printf()` ni `dprintf()`, que escriben por su cuenta, se saltan el buffer y desordenan la salida. `printf()` solo se permite para la ayuda y los errores. Con `-s 1` y con escrituras parciales de 1 byte todo debe seguir funcionando.

- Las ampliaciones se prueban con su propia batería, `minigrep.opcional.json`:

  ```
  ./asosystest.py -i minigrep.opcional.json
  ```

  El comentario de cada prueba empieza por `-n:` o por `-o:` según la opción que evalúa, o por `-n -o:` si usa las dos a la vez. Una ampliación **solo cuenta como hecha si pasan todas sus pruebas**. Las pruebas `-n -o:` solo cuentan si se implementan las dos opciones: en ese caso, si alguna falla, se resta 1 punto por las dos. Los fallos en esta batería no suspenden: solo restan el punto de esa ampliación. Los criterios de calidad del código de «Entrega y evaluación» se aplican también a las ampliaciones.

Ejemplos de ejecución:

```
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "67" -n
2:5678
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "67" -n -v
1:1234
3:90AB
$ echo -e -n "abcabc\nxbbx\nzzz\n" | ./minigrep -r "b+" -o
b
bb
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "[[:alpha:]]+" -o -n
3:AB
$ echo -e -n "axx\nxxa\nb\n" | ./minigrep -r "x*" -o
xx
$ echo -e -n "1234\n5678\n90AB\n" | ./minigrep -r "67" -o -v
ERROR: Las opciones -o y -v son incompatibles
$ echo $?
1
```

En el penúltimo ejemplo, la primera coincidencia de `x*` en `axx` es la cadena vacía del principio de la línea, así que esa línea no produce salida; en `xxa` la primera coincidencia es `xx`, y en `b`, de nuevo vacía.

Depuración

Para depurar el programa con Visual Studio Code, basta con editar el archivo `launch.json` y modificar la entrada `args`. Por ejemplo, para depurar `/usr/bin/echo -en 'REGEX\n' | ./minigrep -r 'REGEX' -s 1048576`, la entrada `args` tendría que tomar el valor: `["-r", "'REGEX'", "-s", "1048576"]`, teniendo que teclear `REGEX` y presionar `Enter` en la entrada estándar una vez iniciado el proceso de depuración. Alternativamente, podríamos enviar la salida del comando `echo` a un archivo con `echo -en 'REGEX\n' > test.txt` para luego usarlo en `args`: `["-r", "'REGEX'", "-s", "1048576", "<", "test.txt"]`.

Entrega y evaluación

- En la entrega se incluirá únicamente el archivo `minigrep.c`.
- El archivo **deberá compilar** conforme a los archivos `tasks.json` o `Makefile` incluidos en `ASO-Semana1.tar.xz`, es decir, con `gcc -ggdb3 -Wall -Werror -Wno-unused -std=c11 minigrep.c -o minigrep`.
- La primera línea de `minigrep.c` deberá ser la directiva para compilar conforme al estándar POSIX: `#define _POSIX_C_SOURCE 200809L`.
- El fichero `minigrep.c` **deberá pasar todos los tests** proporcionados en `minigrep.json` para la utilidad de validación `asosystest.py`. Esa es la parte **obligatoria**.
- Las pruebas de `minigrep.opcional.json` son **optativas**: corresponden a las ampliaciones `-n` y `-o`. No pasarlas no impide aprobar: solo resta 1 punto por cada ampliación a la nota de `minigrep`.

Para aprobar la práctica es imprescindible que el programa pase las pruebas de ejecución, sin que esto signifique que no se puedan detectar otros errores con pruebas adicionales que el profesor haga durante la evaluación.

Se tendrán en cuenta la correcta indentación del código, su estructuración, el manejo de los posibles errores de las llamadas al sistema o funciones de biblioteca, el uso de constantes en lugar de *números mágicos*, así como la correcta gestión de la memoria dinámica. Otro punto importante, y por eso lo volvemos a recordar, es que se penalizará severamente el no usar buffers para las lecturas/escrituras, es decir, leer o escribir byte a byte, pudiendo llegar a suspender la práctica, así como no contemplar lecturas o escrituras parciales si así se pide en el enunciado. Adicionalmente, las ineficiencias que se detecten en el código también podrán ser objeto de penalizaciones. Por ejemplo, reservar y liberar memoria dinámica o inicializar buffers innecesariamente.

Referencias

- Linux Library Functions Manual: regex
- Linux Miscellaneous Information Manual: regex